

Федеральное государственное бюджетное образовательное учреждение высшего образования «Херсонский государственный педагогический университет»
Институт информационных технологий
Направление подготовки: «09.04.04 Программная инженерия»
Специализация «Разработка программно-информационных систем»
Форма обучения: очная
Курс: 2, Магистратура, группа 12-25РПм
Херсонская область - 2026

Дисциплина:
«Процессы и профессиональная практика программной инженерии»

Выполнил:
Дмитрий Сергеевич Трифонов
обучающийся 2-го курса

Проверил:
Алешов Е.В.

ПРАКТИЧЕСКАЯ РАБОТА 2

«Функциональное тестирование программного обеспечения»

I. ЦЕЛЬ И ЗАДАЧИ

1.1. Цель

Целью данной практической работы является изучение принципов функционального тестирования программного обеспечения методом «черного ящика» и освоение способов систематического формирования тестовых вариантов на основе разбиения по классам эквивалентности, анализа граничных значений и диаграмм причинно-следственных связей.

1.2. Задачи

- Изучить особенности функционального тестирования по принципу «черного ящика».
- Рассмотреть категории ошибок, выявляемых функциональным тестированием.
- Освоить способ разбиения входных данных по классам эквивалентности.
- Сформировать классы допустимых и недопустимых входных данных.
- Освоить способ анализа граничных значений.
- Определить тестовые варианты для проверки границ областей допустимых значений.
- Построить дерево разбиений для заданной функциональности.
- Сформировать тестовые варианты для бинарного поиска.
- Рассмотреть применение диаграмм причинно-следственных связей.
- Построить таблицу решений на основе комбинаций причин и следствий.
- Преобразовать столбцы таблицы решений в тестовые варианты.
- Ответить на контрольные вопросы по теме функционального тестирования.

II. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

2.1. Особенности тестирования «черного ящика»

Функциональное тестирование по принципу «черного ящика» рассматривает программное изделие как объект, внутреннее устройство которого не учитывается при формировании тестов. Тестировщик анализирует входные данные и соответствующие им выходные результаты, проверяя выполнение функциональных требований. Основная задача состоит в подборе таких входных данных, которые с высокой вероятностью выявят аномалии поведения программы, и формировании ожидаемых результатов, позволяющих обнаружить дефекты. При этом управляющая структура программы остается вне рассмотрения, а основное внимание уделяется информационной области определения программной системы.

2.2. Категории ошибок, выявляемых функциональным тестированием

- Некорректные или отсутствующие функции.
- Ошибки интерфейса между компонентами или пользователем и программой.
- Ошибки во внешних структурах данных и при доступе к внешним базам данных.
- Ошибки характеристик программного продукта, например несоответствие требуемой емкости памяти.
- Ошибки инициализации и завершения работы программы.

2.3. Разбиение по классам эквивалентности

Разбиение по классам эквивалентности является одним из наиболее распространенных способов функционального тестирования. Входная область данных разделяется на классы эквивалентности, внутри которых программа предположительно должна вести себя одинаково. Для каждого класса выбирается один или несколько характерных тестовых вариантов. Такой подход позволяет сократить количество тестов по сравнению с проверкой каждого возможного значения и одновременно охватить различные варианты поведения программы.

2.4. Правила формирования классов эквивалентности

- Если условие ввода задает диапазон $n...m$, формируются один допустимый класс $\{n...m\}$ и два недопустимых класса: значения меньше n и значения больше m .
- Если условие ввода задает конкретное значение a , формируются один допустимый класс $\{a\}$ и два недопустимых класса: значения меньше a и значения больше a .
- Если условие ввода задает множество конкретных значений $\{a,b,c\}$, формируются один допустимый класс, содержащий указанные значения, и один недопустимый класс, включающий все остальные значения.
- Если условие ввода задает булево значение, формируются два класса: допустимое значение `true` и недопустимое значение `false`.

2.5. Анализ граничных значений

Анализ граничных значений основан на проверке значений, расположенных на границах классов эквивалентности и непосредственно около них. Такой подход связан с тем, что значительная часть ошибок возникает именно при обработке границ допустимых диапазонов. Для диапазона $n...m$ проверяются значения n и m , а также значения непосредственно левее n и непосредственно правее m . Аналогичный принцип применяется к дискретным множествам, выходным данным и границам внутренних структур данных.

2.6. Отличие анализа граничных значений от разбиения по эквивалентности

При разбиении по эквивалентности основное внимание уделяется представительному значению каждого класса допустимых и недопустимых входных данных. Анализ граничных значений концентрируется непосредственно на ребрах классов и значениях около них. Кроме условий ввода, при анализе граничных значений учитываются также границы области вывода, внутренние границы структур данных и крайние элементы упорядоченных наборов.

2.7. Дерево разбиений

Дерево разбиений представляет собой последовательное деление области исходных данных по выбранным критериям. Каждый уровень дерева соответствует определенному условию или специальному требованию, а листья дерева представляют сформированные классы эквивалентности и соответствующие им тестовые варианты. Такой подход позволяет систематически учитывать предусловия, постусловия, специальные случаи и граничные ситуации.

2.8. Диаграммы причинно-следственных связей

Диаграммы причинно-следственных связей используются для формального описания логических условий и соответствующих действий программы. Причины представляют входные условия или классы эквивалентности, а следствия — действия или состояния выходных данных. Между ними устанавливаются логические связи, после чего граф преобразуется в таблицу решений. Каждый столбец таблицы решений соответствует отдельной комбинации условий и преобразуется в тестовый вариант.

2.9. Логические функции и ограничения в диаграммах причин-следствий

- Функция тождество передает состояние причины в соответствующее следствие.
- Функция НЕ инвертирует состояние причины.
- Функция И формирует истинное следствие только при одновременной истинности всех входных причин.
- Функция ИЛИ формирует истинное следствие при истинности хотя бы одной из причин.
- Ограничение Е запрещает одновременную истинность указанных причин.
- Ограничение I требует, чтобы хотя бы одна из указанных причин была истинной.
- Ограничение O требует истинности ровно одной из указанных причин.
- Ограничение R задает зависимость, при которой истинность одной причины требует истинности другой.
- Ограничение M применяется к следствиям и запрещает одновременное появление определенных результатов.

III. МЕТОДЫ И ПОДХОДЫ К ВЫПОЛНЕНИЮ РАБОТЫ

3.1. Общая стратегия функционального тестирования

При выполнении работы программное изделие рассматривается как «черный ящик». Для каждой рассматриваемой функции определяются входные условия, допустимые и недопустимые классы эквивалентности, ожидаемые результаты и потенциально опасные граничные ситуации. После этого формируются минимально необходимые тестовые варианты, обеспечивающие покрытие различных классов поведения.

3.2. Формирование классов эквивалентности для бинарного поиска

- На первом уровне анализируется выполнение предусловий бинарного поиска.
- На втором уровне рассматривается выполнение постусловий: элемент найден или элемент не найден.
- На третьем уровне учитывается количество элементов массива: один элемент, четное количество элементов и нечетное количество элементов больше одного.
- На четвертом уровне анализируются граничные позиции найденного элемента: первый элемент, последний элемент и промежуточный элемент.
- Каждый лист дерева разбиений используется как основа для отдельного тестового варианта.

3.3. Формирование тестов для бинарного поиска

Для проверки бинарного поиска используются упорядоченные массивы различной длины. Отдельно проверяется поиск в одноэлементном массиве, поиск первого, последнего и промежуточного элемента в четном и нечетном массивах, отсутствие искомого элемента, а также нарушение предусловия упорядоченности массива.

3.4. Проверка операции покупки товара

- Определяются предусловия операции: покупатель должен внести деньги, а количество товара должно быть введено корректно.
- Определяется нормальный сценарий, при котором денег достаточно и требуемое количество товара имеется.
- Выделяются ситуации с избыточной и недостаточной суммой денег.
- Отдельно рассматривается ситуация отсутствия необходимого количества товара.
- Проверяются нарушения предусловий: отсутствие денег и некорректный ввод количества товара.

3.5. Построение диаграммы причинно-следственных связей

- Для рассматриваемой функции перечисляются причины и следствия.
- Каждой причине и каждому последствию присваивается уникальный идентификатор.
- Между узлами графа устанавливаются логические связи.
- При необходимости вводятся вторичные причины, возникающие в результате логического преобразования исходных условий.
- Учитываются ограничения совместимости причин и следствий.

3.6. Построение таблицы решений

После построения графа причинно-следственных связей формируется таблица решений. В ней причины представлены как условия, а следствия — как действия или результаты. Для каждого следствия, которое должно принимать значение 1, определяется набор комбинаций причин, приводящих к этому результату. Каждая допустимая комбинация оформляется отдельным столбцом таблицы решений.

3.7. Преобразование таблицы решений в тестовые варианты

Каждый столбец таблицы решений преобразуется в отдельный тестовый вариант. Входные данные теста выбираются таким образом, чтобы реализовать указанную комбинацию причин, а ожидаемый результат определяется соответствующим состоянием следствий. Такой подход позволяет проверить различные логические комбинации входных условий.

IV. ПРАКТИЧЕСКОЕ ВЫПОЛНЕНИЕ И АНАЛИТИЧЕСКОЕ ОСМЫСЛЕНИЕ

4.1. Разбиение по эквивалентности на примере диапазона значений

Для иллюстрации способа рассмотрим входную величину, допустимый диапазон которой составляет от 15000 до 70000. В этом случае формируются три класса эквивалентности: допустимый класс от 15000 до 70000, недопустимый класс значений меньше 15000 и недопустимый класс значений больше 70000. В качестве представителей классов могут быть выбраны значения 40000, 10000 и 80000 соответственно. Такой набор позволяет проверить нормальную обработку допустимого значения и реакцию программы на выход за пределы допустимого диапазона.

4.2. Тестовые варианты для бинарного поиска

- ТВ1 — одноэлементный массив, элемент найден. ИД: $M = [15]$, $Key = 15$. ОЖ.РЕЗ.: $Result=True$; $I=1$.
- ТВ2 — четный массив, найден первый элемент. ИД: $M = [15, 20, 25, 30, 35, 40]$, $Key = 15$. ОЖ.РЕЗ.: $Result=True$; $I=1$.
- ТВ3 — четный массив, найден последний элемент. ИД: $M = [15, 20, 25, 30, 35, 40]$, $Key = 40$. ОЖ.РЕЗ.: $Result=True$; $I=6$.
- ТВ4 — четный массив, найден промежуточный элемент. ИД: $M = [15, 20, 25, 30, 35, 40]$, $Key = 25$. ОЖ.РЕЗ.: $Result=True$; $I=3$.
- ТВ5 — нечетный массив, найден первый элемент. ИД: $M = [15, 20, 25, 30, 35, 40, 45]$, $Key = 15$. ОЖ.РЕЗ.: $Result=True$; $I=1$.
- ТВ6 — нечетный массив, найден последний элемент. ИД: $M = [15, 20, 25, 30, 35, 40, 45]$, $Key = 45$. ОЖ.РЕЗ.: $Result=True$; $I=7$.
- ТВ7 — нечетный массив, найден промежуточный элемент. ИД: $M = [15, 20, 25, 30, 35, 40, 45]$, $Key = 30$. ОЖ.РЕЗ.: $Result=True$; $I=4$.
- ТВ8 — четный массив, элемент не найден. ИД: $M = [15, 20, 25, 30, 35, 40]$, $Key = 23$. ОЖ.РЕЗ.: $Result=False$; I не определено.
- ТВ9 — нечетный массив, элемент не найден. ИД: $M = [15, 20, 25, 30, 35, 40, 45]$, $Key = 24$. ОЖ.РЕЗ.: $Result=False$; I не определено.
- ТВ10 — одноэлементный массив, элемент не найден. ИД: $M = [15]$, $Key = 0$. ОЖ.РЕЗ.: $Result=False$; I не определено.
- ТВ11 — нарушено предусловие упорядоченности. ИД: $M = [15, 10, 5, 25, 20, 40, 35]$, $Key = 35$. ОЖ.РЕЗ.: аварийное сообщение о том, что массив не упорядочен.

4.3. Анализ результатов тестирования бинарного поиска

Сформированный набор из 11 тестовых вариантов покрывает основные классы поведения бинарного поиска. Проверяются одноэлементные, четные и нечетные массивы, поиск первого, последнего и промежуточного элемента, отсутствие искомого значения и нарушение предусловия упорядоченности. При этом тесты ТВ2–ТВ7 позволяют дополнительно проверить обработку граничных позиций элементов массива. Следует отметить, что для нечетного массива из семи элементов последний индекс равен 7, поэтому ожидаемый результат ТВ6 корректно задается как $I=7$.

4.4. Тестовые варианты для операции покупки товара

- ТВ1 — нормальная покупка. ИД: количество товара введено корректно и не превышает имеющегося количества; внесена сумма, равная стоимости покупки. ОЖ.РЕЗ.: выдача запрошенного количества товара.
- ТВ2 — избыточная сумма денег. ИД: количество товара введено корректно; внесена сумма, превышающая стоимость покупки. ОЖ.РЕЗ.: выдача товара и сдачи.
- ТВ3 — недостаточная сумма денег. ИД: количество товара введено корректно и имеется в автомате; внесена сумма меньше стоимости покупки. ОЖ.РЕЗ.: возврат внесенных денег и сообщение «не хватает денег».
- ТВ4 — недостаточное количество товара. ИД: запрошенное количество превышает количество товара в автомате; внесена денежная сумма. ОЖ.РЕЗ.: возврат внесенных денег и сообщение «не хватает продуктов».
- ТВ5 — деньги не внесены. ИД: количество товара введено корректно, но деньги не внесены. ОЖ.РЕЗ.: сообщение «не хватает денег».
- ТВ6 — некорректный ввод количества. ИД: введено нулевое значение или символ, не являющийся цифрой. ОЖ.РЕЗ.: сообщения «ошибка ввода» и «введите количество продуктов»; пользователю предоставляется возможность повторного ввода.

4.5. Диаграмма причинно-следственных связей для расчета оплаты электроэнергии

Для программы расчета оплаты за электроэнергию выделяются четыре основные причины: c_1 — используется средний тариф; c_2 — используется переменный тариф; c_3 — месячное потребление меньше 100 кВт/ч; c_4 — месячное потребление больше или равно 100 кВт/ч. Следствиями являются e_1 — минимальная месячная стоимость, e_2 — процедура А планирования расчета и e_3 — процедура В планирования расчета. Для следствия e_2 дополнительно учитываются вторичные причины, отражающие комбинации исходных условий.

4.6. Таблица решений для расчета оплаты электроэнергии

- Столбец 1: средний тариф и потребление менее 100 кВт/ч → минимальная месячная стоимость.
- Столбец 2: переменный тариф и потребление менее 100 кВт/ч → процедура А планирования расчета.
- Столбец 3: средний тариф и потребление 100 кВт/ч или более → процедура А планирования расчета.
- Столбец 4: переменный тариф и потребление 100 кВт/ч или более → процедура В планирования расчета.

4.7. Тестовые варианты на основе таблицы решений

- ТВ1 — средний тариф, потребление 75 кВт/ч. ОЖ.РЕЗ.: минимальная месячная стоимость.
- ТВ2 — переменный тариф, потребление 90 кВт/ч. ОЖ.РЕЗ.: процедура А планирования расчета.
- ТВ3 — средний тариф, потребление 100 кВт/ч. ОЖ.РЕЗ.: процедура А планирования расчета.
- ТВ4 — переменный тариф, потребление 100 кВт/ч. ОЖ.РЕЗ.: процедура В планирования расчета.

4.8. Аналитическое сравнение методов

Разбиение по классам эквивалентности позволяет уменьшить количество тестов за счет выбора представителей групп входных данных с предполагаемо одинаковым поведением. Анализ граничных значений дополняет этот метод, концентрируясь на местах, где вероятность ошибки особенно высока. Диаграммы причинно-следственных связей используются для более сложных логических зависимостей и позволяют систематически получить комбинации условий через таблицу решений. Таким образом, методы не являются взаимоисключающими и могут применяться совместно.

4.9. Ответы на контрольные вопросы

- **1. Каковы особенности тестирования «черного ящика»?** Программная система рассматривается без учета ее внутренней структуры. Тесты формируются на основании входных данных, функциональных требований и ожидаемых выходных результатов. Основное внимание уделяется поведению программы и ее информационной области.
- **2. Какие категории ошибок выявляет тестирование «черного ящика»?** Некорректные или отсутствующие функции, ошибки интерфейса, ошибки внешних структур данных и доступа к внешним базам данных, ошибки характеристик, а также ошибки инициализации и завершения.
- **3. Поясните суть способа разбиения по эквивалентности.** Входная область разделяется на классы данных, для которых предполагается одинаковое поведение программы. Для каждого класса выбирается характерный тестовый вариант.
- **4. Какие правила формирования классов эквивалентности вы знаете?** Для диапазона формируются один допустимый и два недопустимых класса; для конкретного значения — один допустимый и два недопустимых; для множества конкретных значений — один допустимый и один недопустимый; для булевого условия — классы true и false.
- **5. Поясните суть способа анализа граничных значений.** Метод предусматривает проверку значений непосредственно на границах допустимых областей и рядом с ними. Дополнительно проверяются границы области вывода и внутренние границы структур данных.
- **6. Чем анализ граничных значений отличается от разбиения по эквивалентности?** Разбиение по эквивалентности выбирает представителей классов, а анализ граничных значений специально проверяет ребра классов и соседние с ними значения.
- **7. Что такое дерево разбиений?** Это иерархическая структура последовательного деления области исходных данных по условиям и специальным требованиям. Листья дерева соответствуют классам эквивалентности и тестовым вариантам.
- **8. В чем суть способа диаграмм причин-следствий?** Метод формально связывает входные условия с выходными действиями посредством логического графа, после чего граф преобразуется в таблицу решений и набор тестов.
- **9. Какую структуру имеет граф причинно-следственных связей?** Он содержит узлы причин и следствий, соединенные логическими связями. Причины обычно располагаются в начале цепочки, а следствия — в конце; между ними могут присутствовать промежуточные или вторичные причины.

- **10. Каковы шаги способа диаграмм причин-следствий?** Сначала определяются причины и следствия, затем строится граф, после этого граф преобразуется в таблицу решений, а столбцы таблицы преобразуются в тестовые варианты.
- **11. Какую структуру имеет таблица решений?** Таблица содержит условия и действия. Строки соответствуют причинам, вторичным причинам и следствиям, а столбцы представляют отдельные допустимые комбинации условий и соответствующие действия.
- **12. Как таблица решений преобразуется в тестовые варианты?** Каждый столбец таблицы рассматривается как отдельная комбинация входных условий. Для нее выбираются конкретные входные данные и формулируется ожидаемый результат на основе значений следствий.

V. ВЫВОДЫ

В результате выполнения практической работы были изучены основные принципы функционального тестирования программного обеспечения методом «черного ящика». Рассмотрено отличие функционального тестирования от подходов, ориентированных на внутреннюю структуру программы, а также определены основные категории ошибок, которые могут быть выявлены при проверке внешнего поведения системы. На практике освоено разбиение входных данных по классам эквивалентности, позволяющее уменьшить количество тестовых вариантов за счет группировки данных с одинаковыми ожидаемыми свойствами. Дополнительно применен анализ граничных значений, ориентированный на проверку границ допустимых диапазонов и крайних элементов структур данных. Для бинарного поиска было сформировано дерево разбиений и получен набор из 11 тестовых вариантов, охватывающий основные варианты размера массива, положения искомого элемента, отсутствия элемента и нарушения предусловий. Также рассмотрен способ диаграмм причинно-следственных связей. Для него были определены причины и следствия, сформирована логическая модель, составлена таблица решений и получены четыре тестовых варианта для различных комбинаций тарифов и объема потребления электроэнергии. Практическая работа показала, что систематическое применение методов функционального тестирования позволяет получать обоснованный набор тестовых данных, уменьшать избыточность проверок и одновременно повышать вероятность обнаружения дефектов. Разбиение по эквивалентности, анализ граничных значений и диаграммы причинно-следственных связей целесообразно использовать совместно, поскольку каждый из методов ориентирован на разные аспекты поведения программного продукта.